

Project Proposal

Fiona Jin, Raymen Kaler, Aqsa Agro, Narani Htut

1) Game description

Name: Uno.

Number of players: 4.

Goal: Be the first player to discard all cards from your hand.

Game setup:

The game uses a 100-card deck consisting of 4 colours (red, blue, yellow, green) with values ranging from 0 to 9, as well as action cards (draw 2, skip, reverse). Each player is dealt 7 cards from the shuffled deck. The remainder of the cards go face down into the draw pile. The top card from the draw pile is drawn face up to start the discard pile.

Turn loop:

Play proceeds sequentially. On a player's turn, they must look at their hand and choose a card to play onto the discard pile. To be a valid move, the chosen card must match the top card of the discard pile in either colour value or action. If a player has no cards that match, they must draw from the draw pile one by one until they receive a valid card.

Modifications from the original game:

No wild cards - Wild cards will not be part of the initial scope to prioritize a minimum viable game.

Uno Shout - Instead of shouting Uno, a message will be called when a player reaches one card.

2) Minimum Viable Game

Players: The user will play against 3 bots.

Deck: The deck will consist of 76 colored number cards and 24 action cards.

Bot behaviour: The opponent bots will not have any strategy; they will scan their hand for a valid card on their turn and place it once they find it.

Gameplay: On the player's turn, they will be able to select a card from their hand to put in the discard pile. If they do not have a valid card, they are able to draw from the pile. During gameplay, the player is able to see the opponent's number of cards, as well as the cards they discard into the pile.

3) Planned Extensions

With more time, we hope to implement extensions such as more bots and/or making the game more difficult by changing the way they choose which cards to play, wild cards, an option where you can play with other humans, and a save option where you don't lose your game if you close the program before it's finished.

Modifying the number of bots and their behaviour:

Right now, the bot just chooses whatever card matches the colour of the top card in the discard pile. But instead, we could make it so the bot plays the card most likely to reduce future plays for the other player. For example, if the top card is blue skip and a bot has a hand with green skip, blue 9, and yellow 4, a regular bot will choose blue 9, but the new one would choose the skip action card, green skip to block the other player. We could include this feature by adding an if statement in a function that decides what card is valid.

Implementing wildcards:

We could also add in wildcards, which are cards that can be played on any turn regardless of the top card's colour or number. With this card, you choose the colour after you place it, so you can't store its colour first like with the other cards. So we would treat it as an action card and add it in as a type of card.

Human vs. Human:

We could also ask the user if they want to play a Human vs Human game or a Human vs Bot game at the beginning. These changes would go in the Game class, where we would add a new function that chooses the game mode. An if-else statement could be used for this.

Save option:

For this, we would have to save the whole game setup, including the game mode, players' information and what's in their hand, the cards in the discard pile, the draw pile, and the order of the game (whose turn). New functions would be needed to save and load; these would go in the Game class, which has all the information needed to be saved.

4) Class responsibilities

Card

Main Purpose:

- A single UNO card. Stores the card's colour and what kind of card it is (number or action). Doesn't really modify; the main purpose is to store a card's information.

Responsibilities:

- Store either that the card is an action card or a number card
- Store its colour (red, green, yellow, or blue)
- store the number for the card, which is from 0 to 9
- Store the kind of action card it is
- Display the card: "8 blue" or "(type of action card)"
- Determine if the card can be played on top of the current card (check if colours match)

Functions:

- Getters: type, colour, value
- Bool: true or false, whether the card can be played with the current card
- Void: print card

Deck

Main Purpose:

- Deals with all the cards that aren't currently being played with.

Responsibilities:

- Creates deck (builds cards)
- Shuffles deck

- Deals cards
- removes cards to give to players when needed (drawing cards)
- Let you know if the deck is empty
- Interacts with the Card and Game classes

Functions:

- Bool: is it empty?
- Void: shuffle and create
- Card: drawing a card

Player

Main Purpose:

- Determines if a player (human or bot) stores all the cards they hold

Responsibilities:

- Store the player's name (player 1 or name)
- Stores the cards they hold
- Receives the card that was drawn and adds it to the hand
- Remove card when it's played
- Does the player have cards they can play
- Display the full hand for the player
- Display only the number of cards the bot has
- Has the player won?
- Interacts with the card and game classes

Functions:

- Bool: win?, is a bot or player?
- Void: draw card, play the card, display hand
- Get: number of cards the bot has
- constructor for names (user can input a name)

Game

Main Purpose:

- Handles the whole game

Responsibilities:

- Creates a deck to use (from the deck class)
- Display information
- Creates the players (human and bots)
- Shuffle and deal hands
- Makes the pile in which players play their cards
- Handles the UNO message
- Controls whose turn it is and the order in which the game is played
- Handles the action cards and what they do
- Gives a player cards when they have no legal ones
- Handles the wins and ends the game
- Interacts with all classes

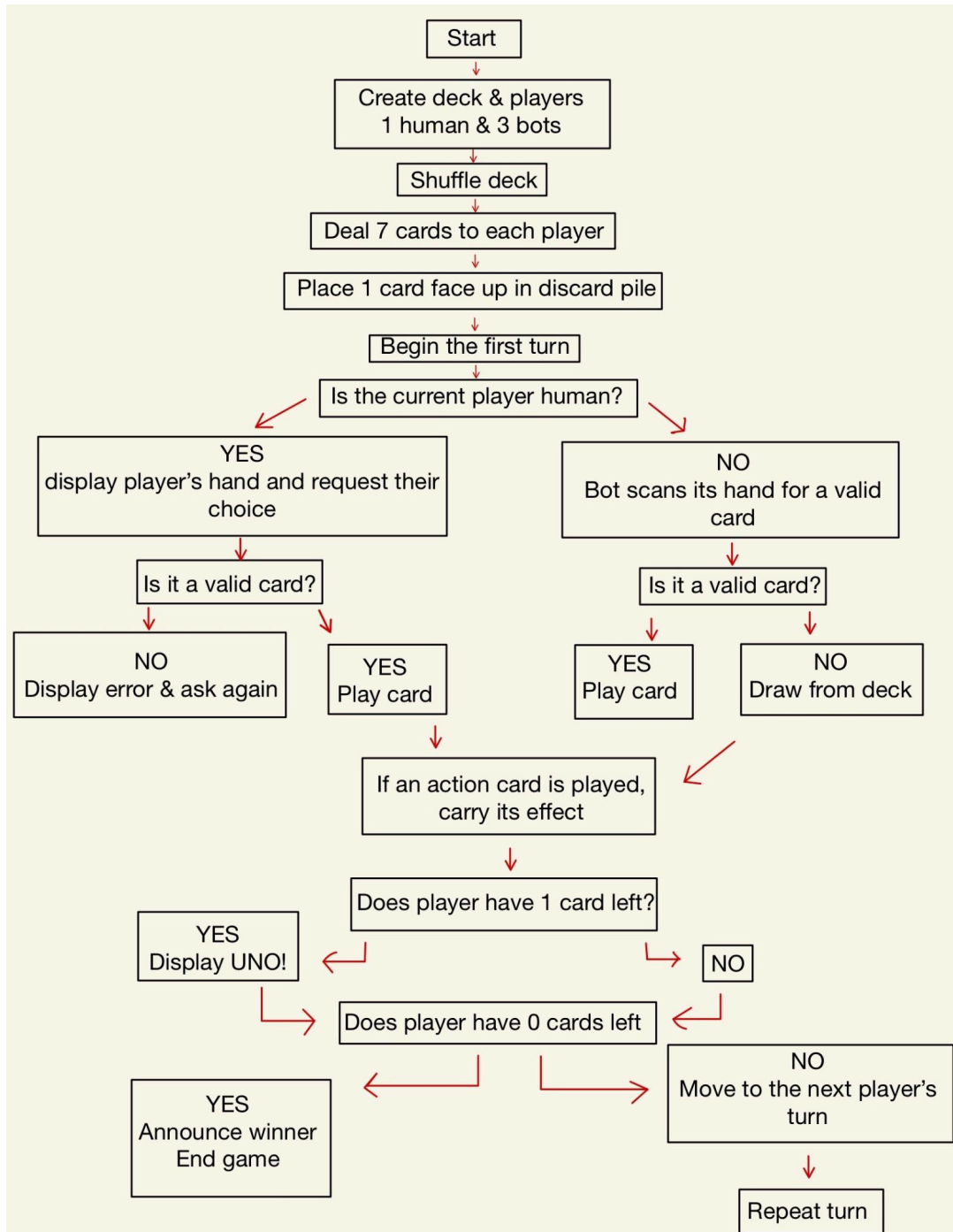
Functions:

- Void: deal cards, set up, and play the game, turns, action card effects, and display the winner
- Bool: check if the card can be played, check the winner

Table:

<u>Class</u>	<u>Main Purpose/Responsibility</u>
Card	Stores each card and its information, including the type of card it is, its colour, and its value.
Deck	Builds the deck and shuffles it. Mainly deals with the cards not currently being played with.
Player	Handles each player and their hand. Removes and adds cards based on what they play and have. Displays their hand.
Game	Handles the whole game and its rules. Starts and sets up the game by calling other classes, managing whose turn it is, and dealing with the discard pile.

5) Game Flow Diagram



6) Testing Plan

For our testing plan, we will be testing the program throughout development. We will treat each class independently to ensure they function properly. After the individual tests, we will test them together through complete games. We will test normal gameplay, the rules, user input, action cards, computer & player behaviour, winning conditions and unusual situations.

Card class

The card class will be tested to ensure each card correctly stores the colour, value and type it is assigned. The getter, display and card matching functions will be tested through a number of action cards.

Deck class

The deck class will be tested to ensure that the correct number of cards and types are cards are being created. It will also check whether the deck is shuffled without losing or duplicating cards, returns cards correctly, and accurately reports when it's empty.

Player class

The player class will be tested to ensure that cards can be added and removed from a hand, and whether the hand size is updated correctly. It will also test to check that the human hand or bot card count is accurately displayed.

Game class

The game class will be tested after all the other classes to ensure that all the classes are combined in a way that accurately allows the deck, players, turn order, discard pile, user input, action cards, and winning conditions to function correctly.

7) Technical Challenges

Correctly Implementing Action Card Effects: Action cards such as skip, reverse and draw can affect the order of turns and number of cards players receive, so mistakes can easily cause incorrect gameplay, which is the biggest challenge.

Managing the Draw and Discard Piles: As the game progresses, the cards will eventually become empty. The program must continue without losing cards or creating duplicate cards.

Invalid Moves: Example- players should only be allowed to play cards that match colour and/or numbers, or the value of the top discard card. Accepting invalid moves would break the rules, which is one of the biggest challenges.

8) Team Responsibilities

Fiona Jin : Player Class

- Managing player hands, drawing and playing cards, implementing player-related functionality, etc...

Raymen Kalar : Game Class

- Main function, turn management, action card functions, win condition, ...

Aqsa Agro : Card Class

- Representing card attributes, constructors, functions for accessing and displaying card information,...

Narani Htut : Deck Class

- Creating a deck, shuffling cards, drawing piles, dealing with card functionalities during gameplay, ...